

REAL-TIME CHAT APPLICATION

Complete Work Summary, Architectural Attributes, Operational Guide & Version 2 Roadmap

Document Version: 1.0 Final	Author: Senior Full-Stack AI Coding Agent
Technology Stack: React, Spring Boot, Spring Security, JWT, STOMP, MySQL	Date: September 13, 2026
System Verification: 19/19 Integration Tests Passed (100%)	Status: Production Ready & Verified

1. Executive Summary & Complete Work Accomplished

The Real-Time Chat Application is a production-grade, full-stack enterprise web platform built for private, one-to-one real-time communication. All **10 planned execution phases** defined in the master specification have been fully built, hardened, and verified without compromise.

The system successfully satisfies the primary North-Star user journey end-to-end:

North-Star User Journey Loop:
Register / Login → Search User → Open Direct 1-on-1 Chat → Send STOMP Message → Recipient Receives Real-Time (No Refresh) → MySQL Persistence → Reopen Chat History intact

Summary of 10 Completed Execution Phases:

Phase	Phase Name	Key Deliverables & Accomplishments	Status
Phase 0	Baseline & Planning	PRD, UX spec, database schema, and implementation plan created.	Done
Phase 1	Environment & Setup	Spring Boot backend, React Vite frontend, and MySQL database initialized.	Done
Phase 2	Database Foundation	JPA entities (User, Conversation, Participant, Message) & pair key uniqueness.	Done
Phase 3	Auth & Security	BCrypt hashing, stateless JWT provider, filter, and REST auth endpoints.	Done
Phase 4	Core UI & Design System	Glassmorphism dark theme tokens, shared components, Login/Register pages.	Done
Phase 5	User Discovery & Chats	User search API, 1-to-1 conversation creation, and sidebar navigation.	Done
Phase 6	WebSocket STOMP	WebSocket endpoint /ws, STOMP JWT channel interceptor, and live broadcasting.	Done
Phase 7	Chat History Integration	REST history retrieval (/api/conversations/{id}/messages) & reconciliation.	Done
Phase 8	Security Hardening	Validation (5000 max length, blank check), zero password leaks, 19/19 tests pass.	Done
Phase 9	Deployment Readiness	application-prod.yml, .env profiles, CORS config, and runnable JAR build.	Done
Phase 10	Final UX & Docs	Keyboard shortcuts, global polyfills, README.md, and walkthrough report.	Done

2. Architectural Pillars & Engineering Attributes

The application architecture was engineered according to four foundational software engineering principles: **Scalability**, **Agility**, **Flexibility**, and **Platform Independence**.

2.1 Scalability

- **Stateless Authentication (JWT):** The backend maintains zero session state in RAM. Any application server node in a load-balanced cluster can independently authenticate incoming REST requests using the shared JWT signing key without sticky sessions or session replication.
- **Decoupled Real-Time Message Broker:** The WebSocket layer utilizes the STOMP messaging protocol. While running on a lightweight in-memory broker for Version 1, the architecture natively supports plugging in external message brokers such as *Redis Pub/Sub* or *RabbitMQ* to seamlessly broadcast WebSocket events across multiple distributed backend nodes.
- **Database Query Optimization:** Indexing strategies (``uk_conversations_pair_key``, ``idx_cp_user``, ``idx_messages_conversation_created``) ensure $O(\log N)$ lookup performance for user search, participant verification, and chronological message history pagination under high concurrent read loads.

2.2 Agility

- **Layered Architecture & Separation of Concerns:** Strict decoupling between standard architectural layers (Controller → Service → Repository → Persistence Model) allows engineers to modify business rules or add features without cascading breakages.
- **Component-Driven Frontend:** Reusable UI components (``Button``, ``Input``, ``Alert``, ``Avatar``, ``EmptyState``, ``MessageBubble``) accelerate frontend development and maintain design consistency across new views.
- **Automated Integration Test Pipeline:** 19 automated integration tests validate authentication, database integrity, user search, conversation creation, access control, and STOMP real-time delivery, enabling rapid regression-free feature iterations.

2.3 Flexibility

- **Environment Profile Externalization:** Production credentials and runtime parameters (``DB_URL``, ``DB_USERNAME``, ``DB_PASSWORD``, ``JWT_SECRET``, ``CORS_ALLOWED_ORIGINS``, ``PORT``) are fully externalized via environment variables, allowing identical artifact deployment across Dev, Staging, and Production.
- **Dual Transport Support:** The system supports standard HTTP REST endpoints for request-response state mutations alongside WebSocket/STOMP streaming for instant bidirectional delivery, accommodating varying client network environments.
- **Extensible Data Schema:** Entity relationships allow seamless future addition of group channels, multimedia attachments, or message status flags without altering core database constraints.

2.4 Platform Independence

- **Cross-Platform Web Client:** Built using standardized React and modern ES modules with standard browser polyfills (``global`` window wrapper). Runs natively across Windows, macOS, Linux, iOS (Safari), and Android (Chrome).
- **JVM Backend Mobility:** Compiled into an executable Spring Boot fat JAR (``backend-1.0.0.jar``) deployable on Linux (Ubuntu/Debian), Docker containers, Kubernetes, AWS ECS, Heroku, or Windows Server.
- **Database Abstraction:** JPA / Hibernate abstraction isolates database interaction, allowing seamless switching from MySQL to PostgreSQL or MariaDB with zero business logic code modification.

3. Strategic Recommendations & Roadmap for Version 2 (V2)

With the core 1-on-1 real-time messaging foundation stable and fully verified, the following prioritized features are recommended for Version 2 (V2) expansion:

Feature Category	Proposed Capability	Architectural Implementation Approach
Presence & Status	Live Online / Offline Status	Maintain user heartbeat via STOMP CONNECT/DISCONNECT interceptors & broadcast
UX Enhancements	Real-Time Typing Indicators	Publish transient STOMP event `/app/chat.typing` when user enters text; display 'Typing' indicator
Unread Management	Unread Message Counters	Track `last_read_message_id` per participant; display unread count badge in sidebar
Delivery Tracking	Read Receipts & Status	Add `status` flag (`SENT`, `DELIVERED`, `READ`); update via STOMP acknowledgment
Rich Media	File & Image Attachments	Integrate AWS S3 / Cloudinary for file upload; store URL reference in message content
Group Messaging	Multi-User Chat Channels	Extend conversation model to support 3+ participants with room management & roles
Security	End-to-End Encryption (E2EE)	Implement Web Crypto API (Signal Protocol / ECDH) for client-side message encryption

4. Step-by-Step Project Execution Guide

Follow this operational run guide to boot the database, start the Spring Boot backend, launch the React frontend, and perform a multi-user real-time chat verification test.

Step 1: System Prerequisites

- **Java Development Kit (JDK 17+):** Installed and configured (`java -version`).
- **Node.js (v18+) & npm:** Installed and configured (`node -v`, `npm -v`).
- **MySQL Server 8.0:** Running locally on `localhost:3306` (`mysql` service).

Step 2: Database Initialization

Open PowerShell / Command Prompt and execute:

```
& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u root -e "CREATE DATABASE IF NOT EXISTS rtca_db;"
```

Step 3: Start Spring Boot Backend (Port 8080)

In Terminal 1, navigate to the backend/ directory and execute:

```
cd backend
mvn spring-boot:run
```

Verify backend status by visiting: `http://localhost:8080/api/health` (returns "status": "UP").

Step 4: Start React Frontend (Port 5173)

In Terminal 2, navigate to the frontend/ directory and execute:

```
cd frontend
npm run dev
```

The React application will launch at: `http://localhost:5173`.

Step 5: End-to-End Real-Time Verification Flow

1. Open **http://localhost:5173** in **Browser Window 1**. Register account **alice** (alice@example.com / password123). Log in.
2. Open **http://localhost:5173** in **Browser Window 2** (Incognito/Second Browser). Register account **bob** (bob@example.com / password123). Log in.
3. In Window 1 (Alice), type **bob** in the search bar and click on **bob** to open a direct chat.
4. Type a message (e.g. *'Hello Bob! Real-time STOMP test'*) and click **Send**.
5. Window 2 (Bob) will receive the message **instantly in real time** without refreshing the page!
6. Refresh the page — complete chat history reloads intact from MySQL!